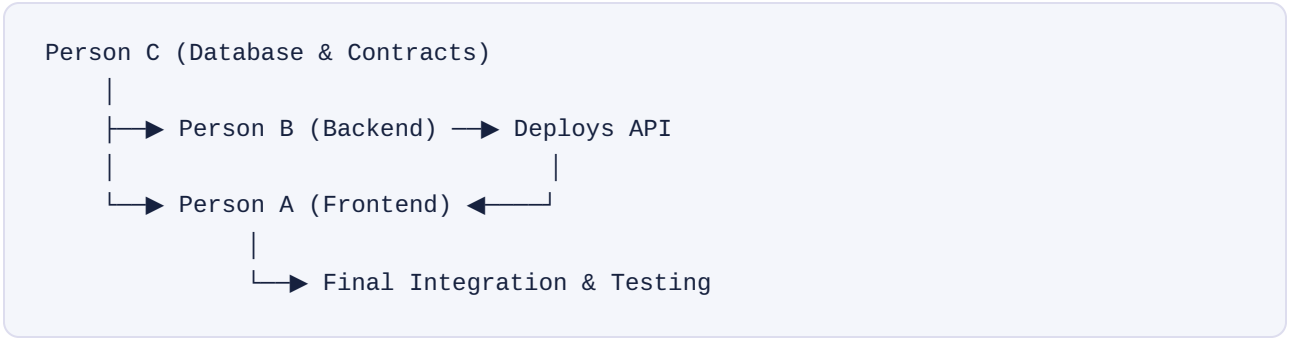
🎙️ Yarn — Task Split

Voice-to-Logbook PWA for SIWES Students | 3-Person Team Allocation

1. Team Roles Overview

Role	Scope	Skills
Person A Frontend	Next.js 14, Web Speech API, PWA, UI/UX	JavaScript / React, CSS
Person B Backend	FastAPI, Gemini integration, Whisper, Encryption	Python
Person C Database & Integration	Supabase setup, RLS, Storage, API contracts, Deployment	SQL, DevOps basics

2. Dependency Flow



Person C starts first (1–2 hours ahead) to create the Supabase project, run the schema, and share credentials + API contract with A and B.

3. Detailed Task Breakdown

Person A Frontend (Next.js 14 + PWA)

- Initialize Next.js 14 project with App Router
- Design & build landing page (`/`) — user creation form
- Build dashboard (`/u/[uuid]`) — log list, date filters, stats
- Build voice recording page (`/u/[uuid]/log/new`) — Web Speech API, waveform UI, audio file upload fallback
- Build log viewer/editor (`/u/[uuid]/log/[id]`)
- Build settings page (`/u/[uuid]/settings`) — custom prompt editor, vocabulary manager, API key input
- Build export page (`/u/[uuid]/export`) — PDF / DOCX download

- PWA setup — `manifest.json`, service worker, mic permissions
- Connect all pages to Person B's live API endpoints

Person B Backend (FastAPI + AI)

- Set up FastAPI project structure, CORS, `.env` config
- Implement Supabase client helper (using Person C's credentials)
- Build user endpoints — `POST/GET/PATCH /api/users`
- Build log endpoints — `POST/GET/DELETE /api/logs`
- Build vocabulary endpoints — `POST/GET/DELETE /api/vocabulary`
- Build `POST /api/process-voice` — Gemini prompt builder + BYOK fallback
- Implement Whisper API fallback for audio file uploads
- Implement Fernet encryption for BYOK key storage
- Confidential mode logic (nullify raw transcript, delete audio)
- Deploy to Render / Railway

Person C Database, Contracts & Deployment

- Create Supabase project & run `01_schema.sql`
- Configure RLS policies (user-level data isolation)
- Create `voice-memos` storage bucket (private, 10 MB max)
- Write & share the **API contract document** (JSON request/response schemas)
- Set up Vercel project for frontend deployment
- Set up Render / Railway for backend deployment
- Manage `.env` secrets distribution (Supabase keys, Gemini key)
- Final integration testing & bug triage
- Export feature backend support (PDF/DOCX generation if needed)

4. Coordination Tips

> 1. Agree on the **API contract first** — Person C drafts a doc listing every endpoint with example request/response JSON. Everyone codes to that contract.
> 2. Use a shared Git repo with branches: `frontend`, `backend`, `database` → merge to `main`.
> 3. Person A can **mock the API** with hardcoded JSON while waiting for Person B to deploy.
> 4. Do a **daily 5-min sync**: "What did you finish? What's blocking you?"

5. Tech Stack Summary

Component	Technology	Cost
Frontend	Next.js 14 (App Router) on Vercel	Free
Backend	FastAPI (Python) on Render	Free
Database	Supabase (PostgreSQL + Storage)	Free tier

STT (Primary)	Web Speech API	Free
STT (Fallback)	OpenAI Whisper API	~$0.006/min
LLM	Google Gemini (gemini-1.5-flash)	Free tier